گزارش کار تمرین عملی

برنامه‌ریزی و بهینه‌سازی هوشمند سفر شهری

نام و شماره دانشجویی: امیررضا مقیمی 403522109

خلاصه مسئله ای که باهاش درگیر هستیم اینه: می‌خوایم از خونه (Home) بریم دانشگاه (Uni). چیزی که باید انتخاب کنیم فقط از کدوم مسیر بریم نیست، ساعت چند راه بیفتیم هم مهمه. چرا؟ چون ترافیک شهر ثابت نیست، مسیری که ساعت ۷:۴۵ عالیه ممکنه ساعت ۸:۳۰ وسط اوج ترافیک گیر کنه. برای همین مسیر و ساعت رو با هم به‌عنوان یه پکیج کامل باید ببینیم و با الگوریتم های Local Search دنبال بهترینش می گردم.

هر استیت یه جفته: x = P, t_start

P : یه مسیر پیوسته و بدون دور از Home تا Uni روی گراف جهت‌دار شهر.

T_start : ساعت حرکت، که فقط می‌تونه از مجموعه‌ی {T = {t_min + k·Δt باشه. تو config.json مقدارها اینان: t_min = 465 (یعنی 07:45)، t_max = 525 (یعنی 08:45) و Δt = 5 دقیقه؛ یعنی 13 ساعت حرکت مجاز داریم.

تابع هدف

زمان عبور از هر یال از رابطه‌ی τ(e,t) = t_base × (1 + C_e(t)) به دست میاد، که C_e(t) ضریب ترافیکه و با گذشت زمان به شکل موج‌های گوسی بالا و پایین می‌ره (اوج ترافیک). زمان کل سفر رو یال به یال و با رعایت FIFO حساب می‌کنیم؛ یعنی زمان رسیدن به هر گره، ساعت حرکت از اون گره رو تعیین می‌کنه. تابع هدف ما f(x) همین زمان کل سفره (به دقیقه) و می‌خوایم کمینه‌ش کنیم. وضعیت‌های نامعتبر (دور، ساعت خارج از بازه و ...) هزینه‌ی بی‌نهایت می‌گیرن.

تابع get_neighbour

این تابع از یه وضعیت، همه‌ی قدم‌های کوچیک و مجاز رو می‌سازه. دو نوع همسایه داریم:

- **جابه‌جایی ساعت حرکت:** ساعت رو به اندازه‌ی یه Δt جلو یا عقب می‌بریم (5 ± t دقیقه). فقط اگه ساعت جدید توی بازه‌ی [t_min, t_max] بمونه قبولش می‌کنیم. ساعت رو هم با round(…, 2) گرد می‌کنم که خطای اعشاری باعث نشه از شبکه‌ی T بیرون بزنه.
- **Detour (تغییر موضعی مسیر):** برای هر دو تا گره‌ی j > i روی مسیر فعلی، بخش بین اون دو تا رو برمی‌دارم و به‌جاش یه مسیر ساده‌ی دیگه بین همون دو گره می‌ذارم. قسمت‌های قبل و بعدش دست‌نخورده می‌مونن.

چطوری مطمئن می‌شیم دور (loop) درست نمی‌شه؟

اینجا دو تا تضمین داریم. اول، موقع پیدا کردن مسیر جایگزین، همه‌ی گره‌هایی که بیرون از بخش انتخابی ان (قبل از i و بعد از j) رو ممنوع می‌کنم، پس مسیر جدید نمی‌تونه از اونا رد بشه. دوم، خود DFS هم گره ای رو که توی همون مسیر نیمه کاره هست دوباره نمیره. در آخر هم هر همسایه یه بار دیگه با problem.is_valid_state چک میشه و همسایه های تکراری و خود وضعیت فعلی حذف می‌شن.

تابع crossover

این تابع از دو والد یه فرزند می سازه:

- اول تقاطع های مشترک رو پیدا می‌کنم: گره‌های میانی‌ای (غیر از Home و Uni) که توی مسیر هر دو والد هستن.
- یکی از اون‌ها رو تصادفی انتخاب می‌کنم و مسیر فرزند رو این‌طوری می‌سازم: **از Home تا اون گره از والد اول + از اون گره تا Uni از والد دوم**.
- ممکنه این ترکیب دور بسازه (یه گره هم توی نیمه‌ی اول باشه هم توی نیمه‌ی دوم). اگه problem.is_valid_path ردش کرد، گره مشترک بعدی رو امتحان می‌کنم.
- ساعت حرکت فرزند میانگین ساعت دو والده، که بعدش به نزدیک‌ترین ساعت مجاز توی T گرد می‌شه تا فرزند همیشه روی شبکه‌ی زمانی بمونه.

- اگه هیچ گره‌ی مشترکی نبود یا همه‌ی ترکیب‌ها دور ساختن، به‌جای برگردوندن یه مقدار نامعتبر، یکی از همسایه‌های والد اول رو برمی‌گردونم. پس خروجی crossover همیشه یه وضعیت معتبره.

توی هر چهار الگوریتم، self.history در هر مرحله کمترین هزینه‌ای که تا اون لحظه دیده شده رو نگه می‌داره. برای همین نمودار همگرایی همیشه نزولی یا ثابته. نتیجه تکرارپذیره: با --seed ثابت، هر بار دقیقاً همون جواب رو می‌گیریم.

Hill Climbing (Steepest Descent + Sideways + Random Restart)

هر بار همه‌ی همسایه‌ها رو می‌سنجه و **بهترین** رو انتخاب می‌کنه. اگه بهتر از وضعیت فعلی بود، می‌ره. اگه برابر بود (با تلورانس 1e-9، چون عددهای اعشاری دقیقا مساوی نمی‌شن)، تا سقف max_sideways تا حرکت افقی مجاز می‌دونیم. حالت‌هایی که قبلاً دیدیم رو دوباره نمی‌ریم که روی دشت دور خودمون نچرخیم. وقتی گیر کرد (نه بهبود، نه حرکت افقی مجاز)، از یه وضعیت تصادفی دوباره شروع می‌کنه؛ بعد از اجرای اول (که از وضعیت اولیه‌ی problem شروع می‌شه) به تعداد max_restarts ری استارت داریم.

Simulated Annealing

از وضعیت اولیه شروع می‌کنه. هر بار یه همسایه‌ی تصادفی می‌گیره؛ اگه بهتر بود قبول می‌کنه، اگه بدتر بود با احتمال $P = \exp(-\Delta E/T)$ قبولش می‌کنه (فیلتر Metropolis). بعد دما رو با $T \leftarrow \alpha \cdot T$ پایین میاره. وقتی تعداد تکرار تموم بشه یا دما از min_temp کمتر بشه می‌ایستیم، و بهترین وضعیتی که دیدیم (نه لزوماً وضعیت آخر) برگردونده می‌شه. اگه $\Delta E = 0$ باشه احتمال پذیرش ۱ می‌شه که درسته.

Local Beam Search

K تا وضعیت (وضعیت اولیه + چندتا تصادفی) رو موازی نگه می‌داره. همسایه‌های هر K تا رو توی یه جای مشترک میریزه و خود beam فعلی رو هم داخلش نگه میداره علتشم این است که جواب هیچوقت بدتر نشه، تکراری‌ها رو حذف می‌کنه که تنوع حفظ بشه و K تا نسخه یه وضعیت نداشته باشیم و بهترین K تا رو برای مرحله‌ی بعد برمی‌داره. وقتی beam جدید با beam قبلی یکی شد یعنی دیگه بهبودی نیست و الگوریتم متوقف میشه.

Genetic Algorithm

جمعیت اولیه = وضعیت اولیه + وضعیت‌های تصادفی. **Fitness** رو 1/(1 + f(x)) تعریف کردم (هر چی زمان سفر کمتر، fitness بیشتر). انتخاب والدین با **Tournament** هست (اندازه‌ی ۳). با احتمال crossover_rate والدین ترکیب می‌شن (وگرنه فرزند کپی والد اوله) و با احتمال mutation_rate فرزند یه جهش می‌گیره که همون یه همسایه‌ی تصادفی از get_neighbors حساب می‌شه. elitism: **Elitism** تا بهترین فرد جمعیت مستقیم می‌رن نسل بعد.

همه‌ی الگوریتم‌ها رو روی ۵ تا ترکیب نقشه/سناریوی پروژه اجرا کردم (toy_graph + rush_hour ، metropolis_graph با سه سناریوی dual_peak_rush_hour و cascading_gridlock و crossover_showcase ، و downtown_grid + flat_plateau). چون SA و GA تصادفی‌ان، هر کدوم رو با ۱۰ seed مختلف (۱ تا ۱۰) اجرا کردم و میانگین گرفتم. «تعداد گام تا همگرایی» یعنی اولین مرحله‌ای که بهترین مقدار نهایی توی history دیده شده (برای HC تعداد حرکت‌ها، برای SA تعداد تکرار، برای Beam تعداد تکرار و برای GA تعداد نسل).

جدول اصلی: سناریوی crossover_showcase

الگوریتم	بهترین f(x*) (دقیقه)	ساعت حرکت بهینه	میانگین زمان اجرا	گام تا همگرایی (میانگین)	تعداد دفعاتی که به بهینه رسید از 20
Hill Climbing	31.01	08:45	0.049	20.8	20
Simulated Annealing	31.01	08:45	0.453	660.4	14
Local Beam Search	31.01	08:45	0.013	2.3	20
Genetic Algorithm	31.01	08:45	0.601	13.5	20

میانگین f(x) روی ۱۰ اجرا هم اینه: HC برابر 31.01 ، Beam برابر 31.01 ، GA برابر 31.01 و SA برابر 31.50. بدترین نتیجه‌ی SA بود؛ یعنی همون دره‌ی محلی ای که گاهی گیرشون میندازه. مسیر بهینه Home → C1 → C2 → T2 → Uni یا Home → C1 → C2 → B2 → Uni بود (هر دو عملاً هم‌زمان‌اند و الگوریتم‌های مختلف یکی از این دو رو برمی‌گردونن).

حالا سوال پیش میاد که چجوری 20 بار ران گرفتم و میانگین حساب کردم، باید بگم با دستور powrshell زیر که میاد 20 بار ران میگیره و در نهایت میانگین زمان اجرا، میانگین گام ها تا همگرایی و تعداد دفعاتی که الگوریتم به حالت بهینه میرسد را محاسبه میکند :

```
$steps=@{}; $wins=@{}; $times=@{}; $worst=@{}; 1..20 | % { $cur=@{}; $n=''; python main.py --algo all --seed $_ --map data/maps/metropolis_graph.json --scenario data/scenarios/crossover_showcase.json | % { if($_ -match '>> Running: (.+)\.\.\.'){$n=$Matches[1].Trim(); if(-not $steps.ContainsKey($n)){$steps[$n]=0; $wins[$n]=0; $times[$n]=0; $worst[$n]=0}} elseif($_ -match 'Convergence Step\s*:\s*(\d+)'){$steps[$n]+=[int]$Matches[1]} elseif($_ -match 'Done in ([\d.]+)s'){$times[$n]+=[double]$Matches[1]} elseif($_ -match 'Total Travel Time:\s*([\d.]+)'){$c=[double]$Matches[1]; $cur[$n]=$c; if($c -gt $worst[$n]){$worst[$n]=$c}} }; $min=($cur.Values | Measure-Object -Min).Minimum; $cur.Keys | % { if([Math]::Abs($cur[$_] - $min) -lt 0.01){$wins[$_]++} } }; $steps.Keys | % { [PSCustomObject]@{ Algorithm=$_; Wins="$($wins[$_])/20"; AvgTime="{0:N3} s" -f ($times[$_]/20); AvgConvSteps=[Math]::Round(($steps[$_]/20), 2); 'Worst f(x)'="{0:N2} min" -f $worst[$_] } } | Format-Table -AutoSize
```

نکته برای اینکه بتونم میانگین تعداد گام ها تا همگرایی رو داشته باشم 2 خط زیر رو در main.py اضافه کردم :

```
conv_step = next((i for i, c in enumerate(history) if abs(c - best_cost) < 1e-6), len(history))
print(f"   Convergence Step : {conv_step}")
```

مقایسه روی همه‌ی سناریوها

اول میانگین f(x) نهایی (دقیقه):

سناریو	هزینه‌ی حالت اولیه	بهینه f(x*)	HC	SA	Beam	GA
toy_graph / rush_hour	96.11	41.77	41.77	41.77	41.77	41.77
metropolis / dual_peak	54.97	31.69	31.69	32.14	31.69	31.69
/ metropolis cascading_gridlock	44.70	31.02	31.02	31.02	31.02	31.02
/ metropolis crossover_showcase	53.78	31.01	31.01	31.29	31.01	31.01
/ downtown_grid flat_plateau	39.30	28.85	28.85	28.85	28.85	28.85

برای این جدول هم تمام الگوریتم هارا بر روی سناریو های مختلف 20 بار ران کردم و میانگین تعداد حالات نوشته شد و با دستور زیر توی ترمینال خروجی گرفتم. نکته : موقع ران کردن این کامند مراقب لپتاپ های عزیز خود باشید (:

```
$scenarios = @(
>>     @{map="data/maps/toy_graph.json"; scn="data/scenarios/rush_hour.json"; name="toy_graph / rush_hour"},
>>     @{map="data/maps/metropolis_graph.json"; scn="data/scenarios/dual_peak_rush_hour.json"; name="metropolis / dual_peak"},
>>     @{map="data/maps/metropolis_graph.json"; scn="data/scenarios/cascading_gridlock.json"; name="metropolis / cascading_gridlock"},
>>     @{map="data/maps/metropolis_graph.json"; scn="data/scenarios/crossover_showcase.json"; name="metropolis / crossover_showcase"},
>>     @{map="data/maps/downtown_grid.json"; scn="data/scenarios/flat_plateau.json"; name="downtown_grid / flat_plateau"}
>> ); $rows = foreach ($s in $scenarios) {
>>     $sumInit = 0.0; $sumOpt = 0.0;
>>     $sumAlgo = @{'Hill Climbing'=0.0; 'Simulated Annealing'=0.0; 'Local Beam Search'=0.0; 'Genetic Algorithm'=0.0};
>>         $seed = $_; $c = @{}; $n = '';
>>         python main.py --algo all --seed $seed --map $s.map --scenario $s.scn | % {
>>             if ($_ -match '\[Initial Cost\]:\s*([\d.]+)') { $sumInit += [double]$Matches[1] }
>>             elseif ($_ -match '>> Running:\s*(.+?)\s*\.\.\.') { $n = $Matches[1].Trim() }
```

```
>>             elseif ($_ -match 'Total Travel Time:\s*([\d.]+)') { $c[$n] = [double]$Matches[1];
$sumAlgo[$n] += [double]$Matches[1] }
>>         };
>>         if ($c.Count -gt 0) { $sumOpt += ($c.Values | Measure-Object -Min).Minimum }
>>     };
>>     [PSCustomObject]@{
>>         'GA'                = "{0:N2}" -f ($sumAlgo['Genetic Algorithm'] / 20);
>>         'Beam'              = "{0:N2}" -f ($sumAlgo['Local Beam Search'] / 20);
>>         'SA'                = "{0:N2}" -f ($sumAlgo['Simulated Annealing'] / 20);
>>         'HC'                = "{0:N2}" -f ($sumAlgo['Hill Climbing'] / 20);
>>         'optimal f(x*)'         = "{0:N2}" -f ($sumOpt / 20);
>>         'initcost' = "{0:N2}" -f ($sumInit / 20);
>>         'senario'           = $s.name;
>>     }
>> }; $rows | Format-Table -AutoSize
```

نکته : برای اینکه این کامند به طور کامل اجرا بشه در main.py دو خط زیر را اضافه کردم و خروجی گرفتم و سپس ان را از main پاک کردم :

```
init_cost = problem.evaluate(problem.generate_initial_state())
print(f"   [Initial Cost]: {init_cost:.2f}")
```

علت اینکه اعداد میانگین در موقعی که جدول 2 را نوشت با اعداد میانگین جدول 1 متفاوت است این است که sa در هر گام به صورت رندوم اجرا میشود و تضمینی برای اینکه بعد از 20 بار ران متفاوت دوباره به همان اعداد برسیم نداریم و احتمال اینکه در هر بار اجرا به جواب متفاوت برسیم وجود دارد.

حالا زمان اجرا (ثانیه) / گام تا همگرایی:

سناریو	HC	SA	Beam	GA
toy_graph / rush_hour	0.012 / 8	0.212 / 525	0.007 / 3	0.430 / 8
metropolis / dual_peak	0.037 / 15	0.464 / 742	0.012 / 2	0.581 / 10
/ metropolis cascading_gridlock	0.042 / 12	0.431 / 736	0.012 / 2	0.586 / 9
/ metropolis crossover_showcase	0.045 / 21	0.447 / 660	0.012 / 2	0.599 / 14
/ downtown_grid flat_plateau	0.024 / 13	0.291 / 481	0.007 / 2	0.570 / 2

برای این جدول هر الگوریتم در 5 سناریوی متفاوت 20 بار ران شده و میانگین زمان اجرا بر روی میانگین تعداد گام ها تا همگرایی نوشته شده، برای این کار هم از دستور زیر استفاده کردم

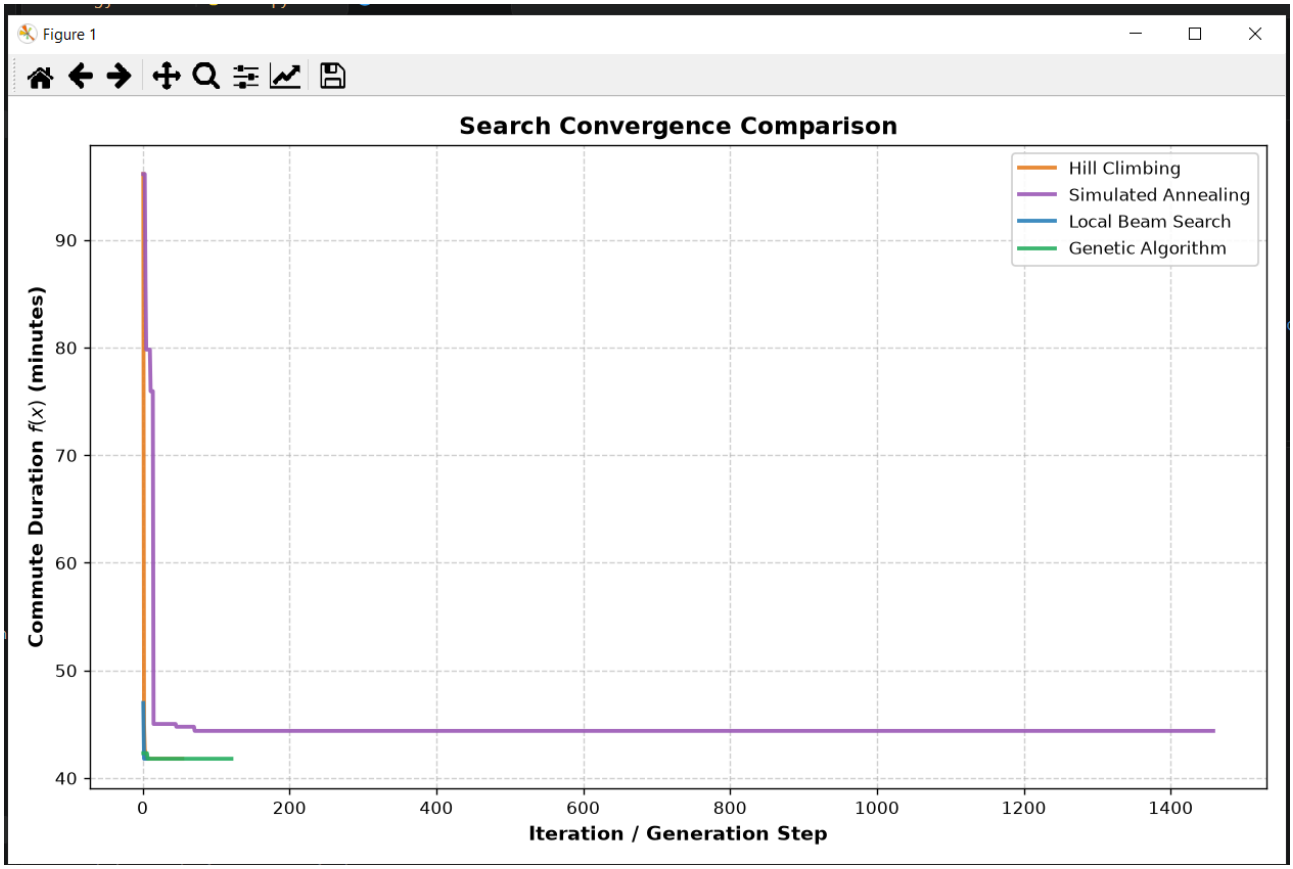
```
$scenarios = @(
>>     @{map="data/maps/toy_graph.json"; scn="data/scenarios/rush_hour.json"; name="toy_graph / rush_hour"},
>>     @{map="data/maps/metropolis_graph.json"; scn="data/scenarios/dual_peak_rush_hour.json"; name="metropolis / dual_peak"},
>>     @{map="data/maps/metropolis_graph.json"; scn="data/scenarios/cascading_gridlock.json"; name="metropolis / cascading_gridlock"},
```

```
>>     @{map="data/maps/metropolis_graph.json";
scn="data/scenarios/crossover_showcase.json"; name="metropolis /
crossover_showcase"},
>>     @{map="data/maps/downtown_grid.json"; scn="data/scenarios/flat_plateau.json";
name="downtown_grid / flat_plateau"}
>>     $st = @{'Hill Climbing'=0; 'Simulated Annealing'=0; 'Local Beam Search'=0; 'Genetic
Algorithm'=0};
>>     $tm = @{'Hill Climbing'=0.0; 'Simulated Annealing'=0.0; 'Local Beam Search'=0.0;
'Genetic Algorithm'=0.0};
>>     1..20 | % {
>>         $n = '';
>>         python main.py --algo all --seed $_ --map $s.map --scenario $s.scn | % {
>>             if ($_ -match '>> Running:\s*(.+?)\s*\.\.\.') { $n = $Matches[1].Trim() }
>>             elseif ($_ -match 'Convergence Step\s*:\s*(\d+)') { $st[$n] += [int]$Matches[1] }
>>             elseif ($_ -match 'Done in ([\d.]+)s') { $tm[$n] += [double]$Matches[1] }
>>         }
>>     };
>>     [PSCustomObject]@{
>>         'GA'     = "{0} / {1:N3}" -f [Math]::Round($st['Genetic Algorithm']/20), ($tm['Genetic
Algorithm']/20);
>>         'Beam'   = "{0} / {1:N3}" -f [Math]::Round($st['Local Beam Search']/20), ($tm['Local
Beam Search']/20);
```

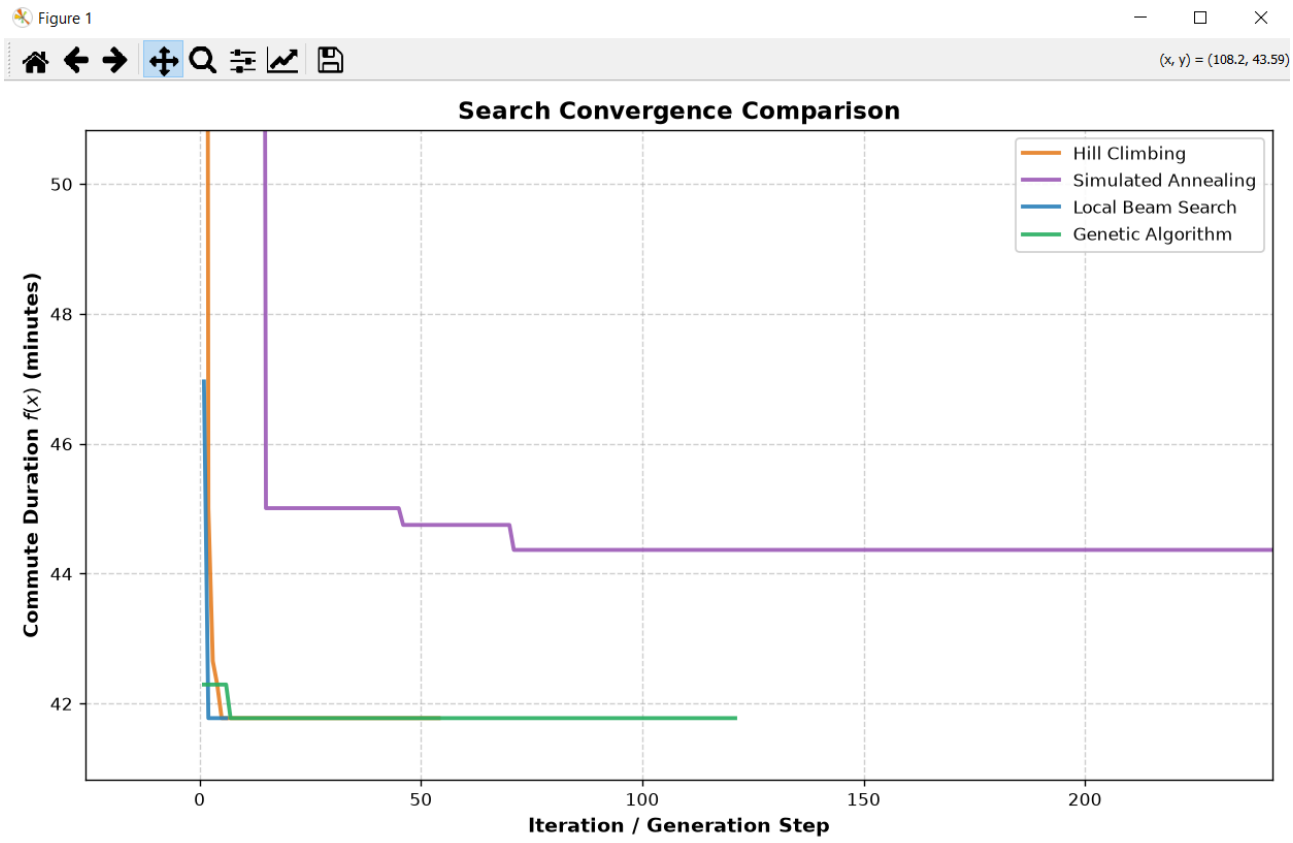
```
>>         'SA'     = "{0} / {1:N3}" -f [Math]::Round($st['Simulated Annealing']/20),
($tm['Simulated Annealing']/20);
>>         'HC'     = "{0} / {1:N3}" -f [Math]::Round($st['Hill Climbing']/20), ($tm['Hill
Climbing']/20);
>>         'سناریو' = $s.name;
>>     }
>> }; $rows | Format-Table -AutoSize
```

دو تا نکته درباره‌ی این جدول‌ها: اولاً ساعت بهینه همه‌جا 08:45 یعنی آخر بازه‌ی مجاز شد. این نشون می‌ده با ترافیک این سناریوها، هر چی دیرتر راه بیفتیم (توی این بازه) بهتره؛ پس ممکنه بهینه‌ی واقعی بیرون از بازه‌ی 07:45 تا 08:45 باشه (البته بازه ی بزرگ تر رو امتحان نکردم، پس این فقط یه حدسه). دوماً «گام» تو الگوریتم‌های مختلف هم‌واحد نیست؛ یه گام Beam یعنی ارزیابی همسایه‌های ۸ تا وضعیت، یه نسل GA یعنی ۵۰ فرد، ولی یه گام SA فقط یه همسایه‌ست. پس مقایسه‌ی گام‌ها رو فقط تقریبی ببینید و زمان اجرا معیار منصفانه‌تره.

نمودار همگرایی



(seed = 42 با) *convergence_comparison*.png

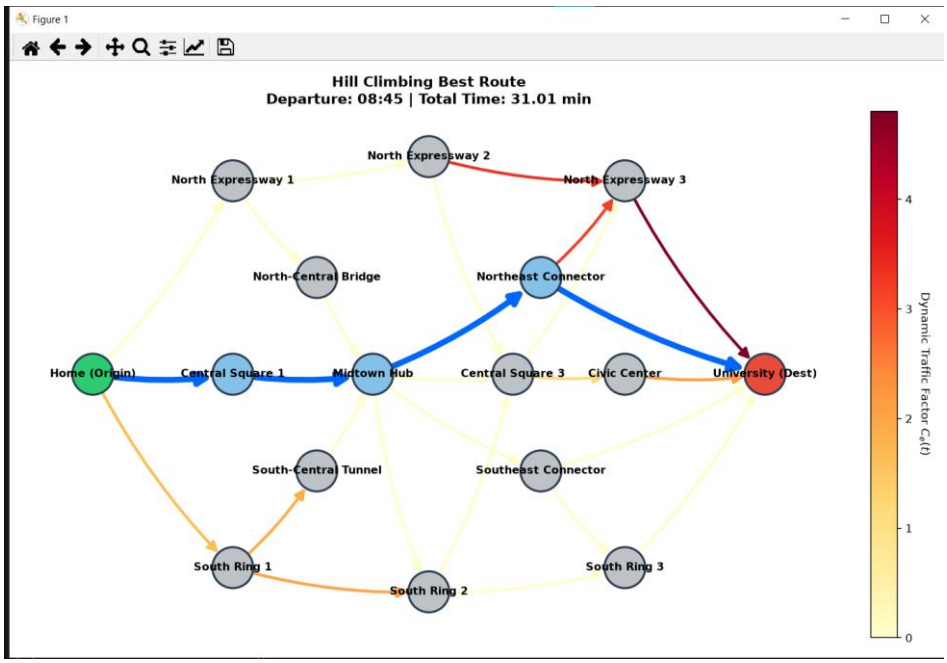


زوم روی گام اول همون نمودار برای دیدن رفتار الگوریتم‌های سریع

تحلیل رفتار روی دشت‌ها، قله‌ها و دره‌های محلی

- **Hill Climbing** و **Beam Search** و **GA** تقریباً توی چند گام اول هزینه رو از 90 و خورده‌ای به حدود 41 می‌رسونن (شکل ۲). HC چون همسایگی خیلی بزرگ داره (حتی کل مسیر رو می‌تونه عوض کنه)، کمتر گیر دره‌ی محلی می‌افته؛ ری‌استارت هم کمک می‌کنه.
- **Simulated Annealing** تو شکل ۱ رفتار کاملاً متفاوتی داره: یه مدت طولانی روی 45 و خورده ای (همون دره‌ی محلی) می‌مونه و تقریباً تا گام حدود 75 تکون نمی‌خوره، بعدش با چند پله به 43 می‌رسه. این پله‌ها یعنی دما اجازه داده از دره‌ی محلی بالا بیاد و چیز بهتری پیدا کنه. مزیتش اینه که از دره می‌تونه فرار کنه؛ بدیش اینه که کند و گاهی ناپایداره (14 بار از 20 بار به بهینه رسید).
- **Beam Search** توی 2 تکرار همگرا شد، چون 8 وضعیت رو هم‌زمان نگه می‌داره و همسایگی بزرگه. هم سریع‌ترین بود و هم هر 20 بار به بهینه رسید.
- **GA** هم توی تمام اجرا ها به بهینه رسید، چون crossover و جهش بهش تنوع می‌دن، ولی ۵۰ فرد در هر نسل باعث می‌شه از بقیه کندتر باشه (حدود 0.5 ثانیه).
- **دشت‌ها:** روی flat_plateau (هزینه‌ها خیلی به هم نزدیک) همه‌ی الگوریتم‌ها به بهینه‌ی 28.85 رسیدن؛ درباره‌ی دلیلش تو پاسخ پرسش ۴ حرف می‌زنم.

مسیر بهینه



بهترین مسیر پیدا شده توسط Hill Climbing (آبی) روی نقشه‌ی metropolis_graph با سناریو crossover_showcase با ساعت حرکت 08:45 و زمان سفر 31.01 دقیقه. رنگ یال‌ها ضریب ترافیک C_e(t) در اون لحظه‌ست.

پاسخ پرسش‌های تحلیلی

پرسش ۱: چرا Gradient Descent اینجا کار نمی‌کنه؟

چرا برای بهینه‌سازی P نمی‌شه ازش استفاده کرد؟ رابطه‌ی $x \leftarrow x - \alpha \nabla f(x)$ به متغیرهایی نیاز داره که پیوسته باشن و بشه کمی تکونشون داد. ولی P یه دنباله‌ی گسسته از گره‌هاست؛ نمی‌شه گفت «مسیر رو ۰٫۳ واحد به سمت گره‌ی C1 ببر». هیچ مفهومی از مشتق نسبت به «انتخاب گره» وجود نداره، و فضای مسیرها هم یه مجموعه‌ی ترکیبیاتیه نه یه فضای هموار. برای همین باید بریم سراغ جستجوی گسسته (همسایگی، ترکیب و ...).

اگه فقط t_start رو بخوایم بهینه کنیم (P ثابت)، چه شرایطی لازمه؟

- t_start باید پیوسته باشه (بازه‌ی [t_min, t_max]) نه فقط مضرب‌های Δt؛ و چون بازه بسته‌ست، باید هر گام رو به بازه برش بدیم (projection).
- C_e(t) باید مشتق‌پذیر باشه. با فرمول $a_{k+1} = a_k + \tau(e_k, a_k)$ داریم $da_{k+1}/dt = (da_k/dt) \times (1 + t_{base} \times C_e'(a_k))$، پس با قاعده‌ی زنجیره‌ای مشتق f و $f = a_m - t_{start}$ رو می‌شه حساب کرد. موج‌های گوسی که توی سناریوها استفاده شده مشتق‌پذیرن، ولی تابع‌های پله‌ای یا قطعه‌ای (مثلاً چراغ قرمز با پرش ناگهانی) نیستن.
- شرط FIFO باید برقرار باشه، یعنی $1 + t_{base} \times C_e' > 0$ (کسی که دیرتر راه می‌افته هیچ‌وقت زودتر نرسه)، وگرنه تابع هدف رفتار عجیب می‌گیره.
- برای رسیدن به کمینه‌ی سراسری، f باید تقریبا محدب باشه یا حداقل یه دره‌ی تکی داشته باشه. با چند اوج ترافیک f (dual_peak) چند دره‌ی محلی داره و Gradient Descent به نزدیک‌ترین دره می‌چسبه.

پرسش ۲: چرا پیکربندی کامل، نه جستجوی تدریجی؟

چون هزینه‌ی هر یال به ساعتی که بهش می‌رسیم بستگی داره و اون ساعت به ساعت حرکت و همه‌ی یال‌های قبلی وابسته‌ست. پس نمی‌شه گفت این بخش از مسیر خوبه بدون اینکه بدونیم کی بهش رسیدیم. مثلا مسیر تو مثال ما اگه ساعت 08:45 حرکت کنه 31 دقیقه طول می‌کشه، ولی همون مسیر با ساعت حرکت دیگه می‌تونه خیلی بدتر باشه (تو آزمایش crossover من یه مسیر ثابت با ساعت 08:40 هزینه‌ی 43.99 داشت و با ساعت 08:10 هزینه‌ش 63.20 شد). اگه مسیر رو قدم به قدم بسازیم باید

ساعت رو از قبل ثابت کنیم و بعد هم باید اون ساعت رو جدا گانه جستجو کنیم؛ در حالی که توی پیکج کامل، ساعت هم یه بعد از فضای جستجوعه و هر دو با هم تغییر می‌کنن.

Dijkstra وابسته به زمان چه محدودیت‌هایی داره و Local Search کجا بهتره؟

- Time-Dependent Dijkstra فقط وقتی دقیقه که شرط FIFO برقرار باشه و ساعت حرکت ثابت باشه. اینجا ساعت متغیره، پس باید یه بار برای هر ساعت مجاز اجراش کنیم
- Dijkstra فرض می‌کنه هزینه جمع پذیره و زیرمسیرهای بهینه، مسیر بهینه می‌سازن. اگه هزینه‌ی وضعیت چیزی مثل جریمه‌ی بی‌ن هایت برای خارج از بازه، محدودیت بدون دور، شرط عبور از یه گره‌ی خاص، یا چند معیار هم‌زمان (زمان + ازدحام + ...) داشته باشه، این فرض میشکنه.
- Local Search فقط به یه تابع f(x) نیاز داره (جعبه سیاه) و هر وقت که متوقفش کنیم یه جواب قابل‌قبول داره (anytime). برای همین توی گراف های بزرگ و هزینه های پیچیده منعطف تره. بدیش اینه که بهینه‌ی سراسری رو تضمین نمی‌کنه.

پرسش ۴: دشت‌ها و Sideways Moves

وقتی ترافیک روی خیلی از یال ها یکنواخته (مثلا ساعت های خارج از اوج، یا وقتی همه‌ی خیابان‌ها یه اوج پهن و هم‌مقدار دارن). اون‌وقت چند مسیر یا چند ساعت حرکت متفاوت تقریبا همون زمان سفر رو می‌دن. سناریوی flat_plateau دقیقاً همینه: همه‌ی اوج‌ها هم‌مقدار و خیلی پهنن (sigma = 40). نشونه‌ش هم تو هزینه‌ی وضعیت اولیه دیده می‌شه: با تغییر ساعت از 07:45 به 07:50، هزینه فقط از 39.30 به 39.13 می‌ره (کمتر از 0.2 دقیقه)؛ یعنی یه شانه (Shoulder) تقریبا مسطح.

max_sideways چه تأثیری داره؟ از نظر تئوری: اگه خیلی کم باشه، HC روی دشت گیر می‌کنه و زود متوقف می‌شه. اگه خیلی زیاد باشه، تا وقتی که سقف تموم نشده روی دشت می‌چرخه و هر گام یه ارزیابی کامل همسایگی خرج داره، پس زمان اجرا بیشتر می‌شه بدون اینکه کیفیت بهتر بشه.

ولی توی آزمایش من چی شد؟ روی HC، flat_plateau بدون ری‌استارت و از ۱۰۰ شروع تصادفی مختلف، با max_sideways برابر 0 ، 1 ، 5 ، 20 و 100 هر ۱۰۰ بار به بهینه‌ی 28.85 رسید، با زمان اجرای تقریبا یکسان (حدود 0.002 ثانیه) و حدود 7.8 گام. یعنی عملا هیچ تاثیری ندید. دلیلش اینه که توی این پروژه حتی روی این سناریو، تابع هدف نسبت به ساعت حرکت همچنان نزولیه (از 39.30 تا 30.25) و همسایگی ما هم خیلی بزرگه؛ پس HC تقریبا همیشه یه همسایه‌ی اکیدا بهتر پیدا می‌کنه و نیازی به

حرکت افقی پیدا نمی‌شه. برای دیدن اثر واقعی max_sideways باید سناریویی داشته باشیم که چند همسایه واقعاً هم‌هزینه باشن.

پرسش ۵: دینامیک برنامه‌ی خنک‌سازی در SA

- **ریسک دمای اولیه‌ی خیلی بالا:** تقریباً هر حرکتی قبول می‌شه و SA به یه قدم زدن تصادفی تبدیل می‌شه که کلی تکرار رو هدر می‌ده توی آزمایش ما چون بهترین دیده‌شده رو نگه می‌داریم جواب نهایی خراب نشد، ولی اگه فقط وضعیت آخر رو می‌گرفتیم خراب‌تر بود و وقت هم زیاد تلف می‌شد.
- **ریسک دمای اولیه‌ی خیلی پایین:** هیچ حرکت بدتری قبول نمی‌شه (تعداد پذیرش 0) و SA عملا مثل Hill Climbing ساده ست و گیر دره ی محلی می‌افته
- **α خیلی کوچیک (0.5، سردسازی ناگهانی / Quenching):** دما خیلی سریع به min_temp میرسه و SA عملا قبل از اینکه جستجو کنه تموم می‌شه، که ممکن است حتی نتیجه اش بدترین میانگین ما باشد. تقریبا هیچ حرکت بدتری هم قبول ممکن است قبول نشود.
- **α خیلی نزدیک به ۱ (0.999):** دما خیلی آروم پایین میاد (مثلا بعد از 3000 تکرار هنوز حدود 6 درجه‌ست) و SA هیچوقت واقعا سرد نمی‌شه. بهترین دیده‌شده خوبه ولی وضعیت آخر هنوز داره می‌گرده یعنی بدون توقف به‌موقع، جستجو همگرا نمی‌شه و باید بهترین دیده‌شده رو ذخیره کنیم.

پرسش ۶: Crossover وقتی والدین هیچ گره‌ی مشترکی ندارن

تابع من چطور باهاش برخورد می‌کنه؟ وقتی هیچ گره‌ی میانی مشترکی نیست، نمی‌شه مسیر رو برید و چسبوند (چون پیوسته نمیمونه و دور یا ناپیوستگی میسازه). پس به‌جای تولید خروجی نامعتبر، یه همسایه‌ی تصادفی از والد اول رو برمی‌گردونم (یه وضعیت معتبر).

چرا ترکیب دو والد خوب می‌تونه فرزند بد بسازه (Epistasis بالا)؟ چون ژن‌ها مستقل از هم نیستن. هزینه‌ی هر بخش از مسیر به ساعتی که بهش می‌رسیم بستگی داره، و اون ساعت به ساعت حرکت و همه‌ی بخش‌های قبلی وابسته است. وقتی بخش‌هایی از دو والد رو به هم می‌چسبونیم و ساعت حرکت رو هم میانگین می‌گیریم، اون بخش‌ها به ساعتی می‌رسن که توی والد خودشون هیچ‌وقت نبودن و ممکنه وسط اوج ترافیک بیفتن.

پرسش ۷: اندازه‌ی فضای حالت و چرا Brute-force جواب نمیده

فرض می‌کنیم گراف |V| گره، میانگین درجه‌ی خروجی b ، میانگین طول مسیر L یال و N_t ساعت حرکت مجاز داریم. هر وضعیت یه مسیر و یه ساعته، پس:

|N_t × X| ≈ N_t × b^L ≥ (تعداد مسیرهای بدون دور)

(چون قید بدون دور تعداد مسیرها رو کم میکنه، یه کران بالای دقیق هم این هست: تعداد مسیرها ≤ (|V|−L−1)! / (|V|−2)! که همون انتخاب مرتب‌شده‌ی L−1 گره‌ی میانی از بین |V|−2 گره‌ی دیگه‌ست.)

چرا برای شهر واقعی Brute-force غیرممکنه؟ یه مثال ساده: شهری با b = 3 خروجی برای هر تقاطع، مسیر 50 یالی و N_t = 120 ساعت حرکت (مثلاً دو ساعت با گام یک‌دقیقه‌ای). اون‌وقت |X| × |120 ≈ 50^3 × 8.6 ≈ 25^10 وضعیت می‌شه. حتی اگه هر ارزیابی یه نانوثانیه طول بکشه (که نمی‌کشه چون هر ارزیابی خودش L یال رو باید حساب کنه)، حدود 8.6 × 16^10 ثانیه یعنی حدود 2.7 میلیارد سال زمان لازم داریم. پس باید با جستجوی محلی از یه نقطه شروع کنیم و فقط همسایه‌های نزدیک رو بررسی کنیم؛ تعداد همسایه‌ها چندجمله ایه (حدود L² × تعداد زیرمسیرها + 2) نه نمایی.